Build Guide: Learning-Bite Crew, Step by Step

Companion to the Assignment Brief | What goes in each file, code skeletons with TODOs, and a checkpoint after every step

How to use this guide. Build in this order: test the built-in tool alone, then build and test the custom tool alone, then the agents, then the tasks, then the crew. Each step ends with a **Checkpoint**; do not move on until your output matches. Only Step 5 needs an OpenAI key, so a failure in Steps 1 - 2 is never an LLM problem.

0. Setup

You need	Check with	Why
Python 3.10 - 3.13	python --version	CrewAI does not install on Python 3.14
OpenAI API key	platform.openai.com -> API keys	Only for Step 5; expected cost for the whole assignment is well under USD 1

```
learning-bite-crew/
├── .env                  # OPENAI_API_KEY=sk-...     (never commit)
├── .gitignore            # .env  .venv/  __pycache__/  output/
├── requirements.txt      # crewai[tools]   python-dotenv
├── test_scrape.py        # Step 1 - run the BUILT-IN tool alone
├── post_tools.py         # Step 2 - the CUSTOM tool + its self-test
├── crew.py               # Steps 3-5 - agents, tasks, crew, kickoff
├── output/               # learning_bite.md is written here
└── notes.md              # Step 6 - experiment observations

python -m venv .venv
source .venv/bin/activate             # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

1. Step 1 - test_scrape.py: see what the researcher will read

Job of this file: call the built-in tool directly, with no agent and no LLM. `tool.run(...)` is exactly what CrewAI calls when an agent uses the tool, so what you see here is what the agent will receive.

```
import sys
from crewai_tools import ScrapeWebsiteTool          # built-in: downloads a page, returns its visible text

DEFAULT_URL = "https://en.wikipedia.org/wiki/Retrieval-augmented_generation"
url = sys.argv[1] if len(sys.argv) > 1 else DEFAULT_URL    # optional URL from the command line

# TODO 1: tool = ScrapeWebsiteTool()                (no website_url here: the agent will pass it per call)
# TODO 2: text = tool.run(website_url=url)
# TODO 3: print the URL, len(text), a token estimate len(text) // 4, then text[:800]
```

Checkpoint 1. `python test_scrape.py` prints a character count in the tens of thousands, a token estimate, and text starting with *The following text is scraped website content:* followed by menu words, then the article. Run it again with `https://en.wikipedia.org/wiki/Vector_database` (your second topic). Record both sizes in notes.md: every character is sent to the model, which is why the research task tells the agent to call the tool **once**.

2. Step 2 - post_tools.py: the custom tool

Job of this file: define a tool the writer agent can call with its draft. A CrewAI custom tool has four parts: (1) a Pydantic **input schema** class, (2) a **name** the agent uses to call it, (3) a **description** the agent reads to decide when to call it, (4) a `_run()` method with the code, which must return a string. The text maths is given below; you write the schema, the tool and the self-test.

```
import re
from typing import Type
from crewai.tools import BaseTool                   # parent class of every custom tool
from pydantic import BaseModel, Field               # describes and validates the tool's input

# ── CONFIGURATION: every rule value lives here and nowhere else ──
MIN_WORDS, MAX_WORDS = 150, 250
MAX_AVG_SENTENCE_WORDS = 20
MIN_READING_EASE = 50          # Flesch Reading Ease, 0-100: 60-70 = plain English, below 30 = academic
MIN_HASHTAGS, MAX_HASHTAGS = 3, 5

# ── GIVEN: text maths in plain Python (no extra library) ──
def count_syllables(word: str) -> int:
    groups = re.findall(r"[aeiouy]+", word.lower())        # each run of vowels ~ one syllable
```

```
    count = len(groups)
    if word.lower().endswith("e") and count > 1:          # silent final 'e' ("make")
        count -= 1
    return max(count, 1)

def text_stats(text: str) -> dict:
    words = re.findall(r"[A-Za-z0-9']+", text)
    sentences = max(len(re.findall(r"[.!?]+", text)), 1)  # at least 1, to avoid dividing by 0
    syllables = sum(count_syllables(w) for w in words)
    n = max(len(words), 1)
    ease = 206.835 - 1.015 * (n / sentences) - 84.6 * (syllables / n)    # standard Flesch formula
    return {"words": len(words), "sentences": sentences, "reading_ease": round(ease, 1)}

class PostQualityInput(BaseModel):
```

```
    # TODO 1: post_text: str = Field(..., description="The complete draft LinkedIn post, including its hashtags.")
    ...

class PostQualityCheckerTool(BaseTool):
    name: str = "Post Quality Checker"
    # TODO 2: description: say what it measures, that it returns VERDICT: PASS / FAIL with fixes,
    #         and "Always use it on a draft before giving the final answer."
    args_schema: Type[BaseModel] = PostQualityInput      # CrewAI validates the agent's arguments against this

    def _run(self, post_text: str) -> str:
        # TODO 3: hashtags = re.findall(r"#\w+", post_text);  body = post_text with hashtags removed
        # TODO 4: stats = text_stats(body); avg_len = round(words / sentences, 1)
        # TODO 5: fixes = []; one line per broken rule, e.g. "Word count is 23; it must be 150-250."
        # TODO 6: report lines:  "Words: N | Sentences: N | Avg sentence: N words"
```

```
        #                       "Flesch Reading Ease: N | Hashtags: N [...]"
        #         return report + "VERDICT: FAIL" + "Fix these:" + "- ..." lines   OR   report + "VERDICT: PASS"
        ...

if __name__ == "__main__":
    tool = PostQualityCheckerTool()
    bad = ("Retrieval-augmented generation is a technique that combines information retrieval with large "
           "language model generation to produce responses grounded in external knowledge bases.")
    print(tool.run(post_text=bad))
    # TODO 7: write your OWN ~200-word post about RAG with 3-5 hashtags, print tool.run(post_text=good)
```

Checkpoint 2. `python post_tools.py` (no key, no internet) prints for the bad sample: *Words: 23 | Sentences: 1 | Avg sentence: 23.0 words*, a negative reading ease, *Hashtags: 0*, **VERDICT: FAIL** and exactly 4 fixes. Your own sample prints **VERDICT: PASS** (edit it until it does: that is the same loop the agent will run). Count the words of your sample in a word processor: the tool must agree.

Two classic bugs. (a) Forgetting the type annotations `name: str =` and `args_schema: Type[BaseModel] =`: BaseTool is a Pydantic model, so unannotated attributes raise an error or are ignored. (b) Returning a dict or a number from `_run()`: return one string, because the agent reads it as text.

3. Step 3 - crew.py, part 1: setup and the three agents

Job of this part: load the key, create one LLM shared by all agents, and define the agents. An agent is role (job title) + goal (what success looks like) + backstory (how it behaves) + tools + llm. The backstory is where you put behaviour rules such as "report only what the page says". One agent is written in full:

```
import os
from pathlib import Path
from crewai import LLM, Agent, Crew, Process, Task
from crewai_tools import ScrapeWebsiteTool          # BUILT-IN tool
from dotenv import load_dotenv
from post_tools import PostQualityCheckerTool       # CUSTOM tool

load_dotenv()                                        # reads OPENAI_API_KEY from .env
Path("output").mkdir(exist_ok=True)

URL = "https://en.wikipedia.org/wiki/Retrieval-augmented_generation"
TOPIC = "Retrieval-augmented generation (RAG)"

llm = LLM(model=os.getenv("OPENAI_MODEL", "gpt-4o-mini"), temperature=0.2)
```

```
researcher = Agent(
    role="Web Researcher",
    goal="Extract accurate, important facts about {topic} from the given web page only.",
    backstory=("You are a careful researcher. You read the source page with your tool and report only what "
               "the page says. If the page does not say something, you leave it out."),
    tools=[ScrapeWebsiteTool()],
    llm=llm, allow_delegation=False, verbose=True,
)

# TODO 1: designer = Agent(role="Learning Designer", ...)   tools=[]  (it only plans)
#         backstory: designs micro-learning for people new to tech; one idea at a time; uses only given facts
# TODO 2: writer = Agent(role="LinkedIn Writer", ...)       tools=[PostQualityCheckerTool()]
#         backstory must include: "You never trust your own word counts: you always run the
```

```
#         Post Quality Checker and fix every problem it reports."
#         add max_iter=10 so a write-check-revise loop cannot run forever
```

Checkpoint 3. `python -c "import crew"` finishes with no error and makes no LLM call (nothing runs until kickoff). An error here is a typo or a missing package, not an agent problem.

4. Step 4 - crew.py, part 2: the three tasks

Job of this part: a task is description (the instructions) + expected_output (what the answer must look like) + agent. {url} and {topic} are placeholders filled at kickoff. `context=[...]` decides which earlier outputs a task can see. The research task is written in full:

```
research_task = Task(
    description=("Use the 'Read website content' tool ONCE on this URL: {url}\n"
                 "From that page only, extract the 8 most important facts about {topic} for a beginner: "
                 "what it is, why it is needed, how it works step by step, benefits, and limits or risks. "
                 "Ignore menus, references and edit links."),
    expected_output=("A one-sentence definition, then a numbered list of exactly 8 facts. Each fact is one or two "
                     "plain sentences and ends with the page section it came from, e.g. (section: Process)."),
    agent=researcher,
)

# TODO 3: outline_task = Task(... agent=designer, context=[research_task])
#   description: using ONLY the research facts, plan a learning bite about {topic} for freshers with six
#                labelled parts: hook question; definition under 25 words; 3 key points; one workplace
#                example; one misconception + correction; one-line call to action
```

```
#   expected_output: "A bullet-point outline with the six labelled parts above. No full paragraphs."

# TODO 4: write_task = Task(... agent=writer, context=[outline_task, research_task],
#                           output_file="output/learning_bite.md")
#   description: write the post from the outline; call the Post Quality Checker with the FULL post;
#                if VERDICT: FAIL, fix every listed problem and check again; repeat until PASS (at most
#                3 checks); add no facts that are not in the research
#   expected_output: the final post, a line '---', then the last checker report copied exactly
#                    (it must end in VERDICT: PASS)
```

Why the writer also gets research_task in context: the outline is a summary; without the original facts the writer fills gaps from its own memory, which is exactly the invented-facts problem from the scenario.

5. Step 5 - crew.py, part 3: assemble and run

```
crew = Crew(agents=[researcher, designer, writer],
            tasks=[research_task, outline_task, write_task],
            process=Process.sequential, verbose=True)

if __name__ == "__main__":
    result = crew.kickoff(inputs={"url": URL, "topic": TOPIC})    # fills {url} and {topic} in every task/agent
    print(result.raw)                                             # plain text of the last task's output
    print("Token usage:", result.token_usage)                     # note this in notes.md
```

Checkpoint 5. `python crew.py` shows, in the verbose log: Agent 1 with a *Tool Execution* block for **Read website content** and the URL as input (once); Agent 2 with no tool calls; Agent 3 with at least one **Post Quality Checker** call. `output/learning_bite.md` exists and ends with a checker report that says **VERDICT: PASS**. Re-run the checker yourself on the saved post (paste it into your self-test): the numbers must match the report. Rename the file to `learning_bite_rag.md`, set URL to the Vector database page and TOPIC to "Vector databases", run again, and keep both files.

6. Step 6 - experiments for notes.md

ID	Change exactly one thing	Record
E1	Remove PostQualityCheckerTool from the writer; change Task 3 to "Write a 150-250 word post and state its word count at the end."	The word count the model claims vs the real count from your tool. Repeat 2 times. Restore afterwards.
E2	Change the checker's description to "A tool."	Did the writer still call it? How many times? Did the post still pass?
E3	Remove research_task from write_task's context	Compare the two posts: does any fact appear that is not in the 8 facts?
E4	No change: compare the two topic runs	Token usage of each run, and how many checker calls the writer needed before PASS

7. Troubleshooting

Symptom	Cause	Fix
pip install fails on crewai	Python 3.14	Create the venv with Python 3.10 - 3.13
Scrape returns a short 'Access denied' / empty text	The site blocks scripts	Use the Wikipedia URLs given; check with test_scrape.py first
ValueError ... resolves to private/reserved IP	crewai-tools blocks localhost/internal URLs by default	Use a public URL (do not bypass this for the assignment)
AuthenticationError / 401	Key missing or wrong in .env	.env next to crew.py; no quotes or spaces around the key
Agent says the tool does not exist	Tool name in the task text differs from the tool's name	Use the exact names 'Read website content' and 'Post Quality Checker'
Writer stops at 'Maximum iterations reached'	Draft never passed within max_iter	Read the FAIL reasons in the log; make the outline shorter or raise max_iter
Final file paraphrases the checker report	expected_output not strict enough	Say 'copied exactly' and 'must end in VERDICT: PASS'

8. Submission checklist

- Checkpoints 1, 2, 3 and 5 pass from a fresh clone and a new venv.
- Both output posts end with VERDICT: PASS, and every fact can be found on its source page.
- Screenshots: Checkpoint 2 output, both tool-call blocks in the verbose log, one final post.
- notes.md: E1 - E4 with real numbers. .env, .venv/ and output/ are not committed (submit the posts separately).